AIMERZ.ai
Aim.Act.Achieve

NOTES

Collections Framework/API - 2

Queue, Concrete classes
Exploring Iterators
Concurrent Modification (Fail-Fast Vs Fail-Safe)

Queue Interface: A Queue represents a collection designed for holding elements before processing. The typical queue is FIFO (First In, First Out).

Common Methods in Queue:

queue.add(10); // Adds an element

queue.offer(20); // Adds an element, returns false if failed

queue.poll(); // Retrieves and removes the head of the queue

queue.peek(); // Retrieves, but does not remove, the head

Key Implementation classes PriorityQueue, LinkedList and ArrayDeque

PriorityQueue in Java

The PriorityQueue class in Java is part of the java.util package. It is designed to hold elements in a specific order, determined by their natural ordering or a custom comparator provided at the time of creation.

Real-World Example

- ➔ **Task Scheduling**: Tasks are executed based on priority.
- ➔ **Job Queue in Printers**: High-priority documents are printed first.
- ➔ **Event Management**: Processing events in priority order.

Features of PriorityQueue

- **Priority-Based Ordering**: 1. Elements are arranged based on their priority (natural order for numbers/strings or a custom order for objects). 2. The smallest (or highest-priority) element is always at the head of the queue.
- **Implements Queue Interface**: Extends AbstractQueue and implements the Queue interface, providing queue functionalities.
- **Not Thread-Safe**: Not synchronized by default. Use PriorityBlockingQueue for thread safety.
- **No Null Elements Allowed**: Does not permit null values.
- **Dynamic Sizing**: Automatically grows as elements are added.
- **Allows Duplicates**: Unlike sets, duplicates are permitted in the queue.
- **Heap-Based Data Structure**: Internally implemented using a **binary heap**.
- **Retrieval is Constant Time (O(1))**: Accessing the highest-priority element (head) is constant time.
- **Insertion and Deletion in Logarithmic Time (O(log n))**: Efficient for adding and removing elements.
- **Not Ordered Like a List**: Iteration order does not reflect element priority or insertion order.

Constructors of PriorityQueue:

- **PriorityQueue()**: Creates an empty priority queue with the default initial capacity (11) and natural ordering.
- **PriorityQueue(int initialCapacity)**: Specifies the initial capacity.
- **PriorityQueue(int initialCapacity, Comparator<? super E> comparator)**: Specifies initial capacity and comparator.
- **PriorityQueue(Collection<? extends E> c)**: Creates a priority queue containing the elements of the specified collection.
- **PriorityQueue(PriorityQueue<? extends E> c)**: Creates a priority queue containing the elements of the specified priority queue.
- **PriorityQueue(SortedSet<? extends E> c)**: Creates a priority queue containing the elements of the specified sorted set.

When to use PriorityQueue

PriorityQueue: Elements are ordered based on their natural order or a comparator. Used when priority-based processing is required.

PriorityQueue Example:

```java
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        // Create a PriorityQueue of Integer type
        PriorityQueue<Integer> queue = new PriorityQueue<>();

        // Add elements to the queue
        queue.add(20);
        queue.add(10);
        queue.add(30);

        // Elements are ordered based on their natural order
(smallest first)
        System.out.println("Queue: " + queue);

        // Peek the head of the queue
        System.out.println("Head of the queue: " + queue.peek());

        // Poll (retrieve and remove) the head of the queue
        System.out.println("Polled: " + queue.poll());

        // After polling, check the new head
        System.out.println("New head of the queue: " + queue.peek());
    }
}
```

Output:

```java
Queue: [10, 20, 30]
Head of the queue: 10
Polled: 10
```

```
New head of the queue: 20
```

Comparison with Other Queues

Feature	Queue	Deque	PriorityQueue
Order Maintained	FIFO	FIFO or LIFO	Based on priority
Duplicates Allowed	Yes	Yes	Yes
Null Elements	Not allowed	Not allowed	Not allowed
Performance	Simple operations	Efficient for both ends	Efficient for priority tasks

LinkedList (used as a Queue):

The LinkedList class in Java implements the following interfaces:

- ➔ **List Interface**: Enables LinkedList to function as a sequential list where elements can be accessed by index.
- ➔ **Queue Interface**: Allows LinkedList to function as a queue (FIFO — First In, First Out).

We can verify this from its definition in Java's **API documentation**:

```Java
public class LinkedList<E> extends AbstractSequentialList<E>
        implements List<E>, Deque<E>, Cloneable, Serializable
```

The Queue interface is extended by Deque (double-ended queue), which is implemented by LinkedList.

When to use LinkedList?

LinkedList: When we need a simple FIFO (First In First Out) or LIFO (Last In First Out) queue or as List (for Index based accessing)

Code Proof with Examples

Using LinkedList as a List

```
Java
import java.util.LinkedList;
import java.util.List;

public class LinkedListAsList {
    public static void main(String[] args) {
        List<String> list = new LinkedList<>();

        // Adding elements
        list.add("A");
        list.add("B");
        list.add("C");

        // Accessing elements
        System.out.println("Element at index 1: " + list.get(1)); // Output:
B

        // Removing element
        list.remove(0); // Removes "A"
        System.out.println("List after removal: " + list); // Output: [B, C]
    }
}
```

Above Code Explanation:
Here, LinkedList behaves like a sequential list where elements are accessed by index.

Using LinkedList as a Queue

```
Java
import java.util.LinkedList;
import java.util.Queue;
```

```java
public class LinkedListAsQueue {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();

        // Adding elements to the queue
        queue.offer(10);
        queue.offer(20);
        queue.offer(30);

        // Accessing and removing elements (FIFO order)
        System.out.println("Removed: " + queue.poll()); // Output: 10
        System.out.println("Next Element: " + queue.peek()); //
Output: 20
        System.out.println("Queue after removal: " + queue); //
Output: [20, 30]
    }
}
```

Output:

```java
Removed: 10
Next Element: 20
Queue after removal: [20, 30]
```

Above Code Explanation:
Here, LinkedList behaves like a queue. The element inserted first (10) is the first to be removed (FIFO).

Adding/Removing from Both Ends (Deque Functionality)

```java
import java.util.Deque;
import java.util.LinkedList;

public class LinkedListAsDeque {
    public static void main(String[] args) {
```

```
        Deque<String> deque = new LinkedList<>();

        // Adding elements to both ends
        deque.addFirst("Front");
        deque.addLast("Back");

        // Accessing and removing from both ends
        System.out.println("Removed from front: " + deque.removeFirst()); //
Output: Front
        System.out.println("Removed from back: " + deque.removeLast());   //
Output: Back
    }
}
```

Above Code Explanation:
Here, LinkedList acts as a deque (double-ended queue), supporting operations at both ends efficiently.

ArrayDeque in Java

The ArrayDeque (short for Array Double-Ended Queue) is a resizable array-based implementation of the Deque interface. It allows elements to be added or removed from both ends efficiently, making it a highly versatile and performance-oriented data structure.

Features of ArrayDeque

- ➔ **Double-Ended Operations**: 1. Supports insertion and removal from both the front and back. 2. Ideal for implementing **stacks** and **queues**.
- ➔ **Resizable Array**: 1. Internally uses a resizable array to store elements. 2. Automatically grows as needed when the capacity is exceeded.
- ➔ **Efficient Performance**: 1. O(1) time complexity for adding/removing at both ends. 2. Faster than LinkedList for deque operations due to better cache locality.

- **No Null Elements**: Does not allow null values as they are used to indicate the absence of elements.
- **Thread Safety**: Not synchronized. Use Collections.synchronizedDeque or manually synchronize for multi-threaded environments.
- **Replacement for Stack and Queue**: Preferred over Stack (from the legacy collection) and LinkedList for stack and queue implementations.

Constructors Used in ArrayDeque with Load Factor

- **ArrayDeque()**: Creates an empty deque.
- **ArrayDeque(int initialCapacity)**: Creates an empty deque with the specified initial capacity.

Let's see the, **Common Methods of ArrayDeque**

Method	Description
addFirst(E e)	Inserts the specified element at the front of the deque.
addLast(E e)	Inserts the specified element at the end of the deque.
offerFirst(E e)	Inserts the element at the front, returns false if the deque is full.
offerLast(E e)	Inserts the element at the end, returns false if the deque is full.
removeFirst()	Removes and returns the element at the front. Throws exception if empty.
removeLast()	Removes and returns the element at the end. Throws exception if empty.

pollFirst()	Retrieves and removes the first element, or returns null if the deque is empty.
pollLast()	Retrieves and removes the last element, or returns null if the deque is empty.
getFirst()	Retrieves but does not remove the first element. Throws exception if empty.
getLast()	Retrieves but does not remove the last element. Throws exception if empty.
peekFirst()	Retrieves the first element, or returns null if empty.
peekLast()	Retrieves the last element, or returns null if empty.

Real-World Analogy

Imagine a **line at a ticket counter** where people can join or leave from either the front or the back:

- ➔ If you're in a rush, you might join at the **front** (addFirst).
- ➔ If you're patient, you might join at the **back** (addLast).
- ➔ The ticket issuer can serve people from either end (removeFirst or removeLast).

When to Use ArrayDeque?

- ➔ Need a resizable **stack** or **queue**.
- ➔ Require frequent insertions/removals at both ends.
- ➔ Prefer better performance over `LinkedList`.

Example of ArrayDeque: Basic Usage

```Java
import java.util.ArrayDeque;

public class ArrayDequeExample {
    public static void main(String[] args) {
        ArrayDeque<String> deque = new ArrayDeque<>();
```

```
        // Adding elements
        deque.addFirst("Front");
        deque.addLast("Back");

        System.out.println("Deque: " + deque); // Output: [Front,
Back]

        // Accessing elements
        System.out.println("First Element: " + deque.getFirst()); //
Output: Front
        System.out.println("Last Element: " + deque.getLast());   //
Output: Back

        // Removing elements
        deque.removeFirst();
        System.out.println("After removeFirst(): " + deque); //
Output: [Back]
    }
}
```

Output:

```
Java
Deque: [Front, Back]
First Element: Front
Last Element: Back
After removeFirst(): [Back]
```

Using ArrayDeque as a Stack

```
Java
import java.util.ArrayDeque;
```

```java
public class StackUsingArrayDeque {
    public static void main(String[] args) {
        ArrayDeque<Integer> stack = new ArrayDeque<>();

        // Push elements
        stack.push(10);
        stack.push(20);
        stack.push(30);

        System.out.println("Stack: " + stack); // Output: [30, 20,
10]

        // Pop elements
        System.out.println("Popped: " + stack.pop()); // Output: 30

        // Peek at the top
        System.out.println("Top Element: " + stack.peek()); //
Output: 20
    }
}
```

Output:

```java
Stack: [30, 20, 10]
Popped: 30
Top Element: 20
```

Why ArrayDeque for Stack?

➔ Unlike Stack (from the legacy collection), ArrayDeque is faster and more modern.

Using ArrayDeque as a Queue:

```java
import java.util.ArrayDeque;

public class QueueUsingArrayDeque {
```

```java
    public static void main(String[] args) {
        ArrayDeque<String> queue = new ArrayDeque<>();

        // Offer elements to the queue
        queue.offer("A");
        queue.offer("B");
        queue.offer("C");

        System.out.println("Queue: " + queue); // Output: [A, B, C]

        // Poll (FIFO order)
        System.out.println("Polled: " + queue.poll()); // Output: A
        System.out.println("Queue after poll: " + queue); // Output:
[B, C]
    }
}
```

Output:

```java
Queue: [A, B, C]
Polled: A
Queue after poll: [B, C]
```

Why ArrayDeque for Queue?

➔ Faster and more efficient than LinkedList for queue operations.

Let's **Explore Iterators**:

In Java, there are several ways to iterate over elements of collections like **List, Set, Queue,** and **Map.** Each approach has its pros and cons, depending on the collection type and the operations we're performing.

Here are 5 common ways to iterate over elements in all the collection classes we discussed earlier:

- ➔ Using **For-each loop (Enhanced for loop)**
- ➔ Using **Iterator**
- ➔ Using **ListIterator (for Lists)**
- ➔ Using **DescendingIterator**
- ➔ Using **Streams (will be introduced in Java 8)**
- ➔ Using **for loop with index (for List)**

Comparison Table for Iterations

Classes	For Loop	Enhanced 'for loop'	Iterator	List Iterator	Descending Itrator	Enumerator
ArrayList	✔	✔	✔	✔	✘	✘
LinkedList	✔	✔	✔	✔	✔	✘
ArrayDeque	✘	✔	✔	✘	✔	✘
PriorityQueue	✘	✔	✔	✘	✘	✘
TreeSet	✘	✔	✔	✘	✔	✘
HashSet	✘	✔	✔	✘	✘	✘
LinkedHashSet	✘	✔	✔	✘	✘	✘

Let's begin with, **For-each Loop (Enhanced For Loop)**

This is a simple and clean way to iterate over collections. It works for any collection that implements the Iterable interface (like List, Set, Queue, etc.).

Example: For-each with ArrayList

```Java
import java.util.ArrayList;

public class ForEachExample {
      public static void main(String[] args) {
      ArrayList<String> fruits = new ArrayList<>();
      fruits.add("Apple");
      fruits.add("Banana");
      fruits.add("Orange");

      // Using enhanced for loop
      for (String fruit : fruits) {
            System.out.println(fruit);
      }
      }
}
```

Output

```Java
Apple
Banana
Orange
```

Example for-each with HashSet

```Java
import java.util.HashSet;

public class ForEachWithSet {
      public static void main(String[] args) {
          HashSet<Integer> numbers = new HashSet<>();
          numbers.add(10);
          numbers.add(20);
          numbers.add(30);

          // Enhanced for loop
          for (Integer num : numbers) {
```

```
                System.out.println(num);
            }
        }
}
```

Output:

```
Java
20
10
30
```

Using Iterator: Iterator is an Interface and, The Iterator interface provides methods to iterate over any collection and is more flexible than the enhanced for loop. It allows for safe element removal during iteration.

- ➔ hasNext() - returns true if there are more elements.
- ➔ next() - returns the next element.
- ➔ remove() - removes the last element returned by next() (optional).

Example: Iterator with LinkedList

```
Java
import java.util.Iterator;
import java.util.LinkedList;

public class IteratorExample {
    public static void main(String[] args) {
        LinkedList<String> animals = new LinkedList<>();
        animals.add("Dog");
        animals.add("Cat");
        animals.add("Elephant");
```

```java
        // Creating an iterator
        Iterator<String> iterator = animals.iterator();

        // Iterate using the iterator
        while (iterator.hasNext()) {
            String animal = iterator.next();
            System.out.println(animal);

            // Remove an element (optional)
            if ("Cat".equals(animal)) {
                iterator.remove();
            }
        }

        // Print after removal
        System.out.println("After removal: " + animals);
    }
}
```

Output:

```
Java
Dog
Cat
Elephant
After removal: [Dog, Elephant]
```

Using ListIterator (for Lists): ListIterator is an extension of Iterator with additional methods that allow bidirectional traversal (forward and backward) and modification of elements. It is only applicable to List implementations like ArrayList and LinkedList.

- ➔ hasPrevious() - checks if there's a previous element.
- ➔ previous() - returns the previous element.
- ➔ add() - adds an element before the current cursor position.
- ➔ set() - replaces the last element returned by next() or previous().

Example: ListIterator with ArrayList

```Java
import java.util.ArrayList;
import java.util.ListIterator;

public class ListIteratorExample {
    public static void main(String[] args) {
        ArrayList<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");

        // Creating a ListIterator to traverse in both directions
        ListIterator<String> listIterator = fruits.listIterator();

        System.out.println("Forward direction:");
        while (listIterator.hasNext()) {
            System.out.println(listIterator.next());
        }

        System.out.println("\nBackward direction:");
        while (listIterator.hasPrevious()) {
            System.out.println(listIterator.previous());
        }

        // Modifying elements
        listIterator.next();
        listIterator.set("Mango"); // Replaces "Apple" with "Mango"

        System.out.println("After modification: " + fruits);
    }
}
```

Output:

```
Java
Banana
Cherry

Backward direction:
Cherry
Banana
Apple
After modification: [Mango, Banana, Cherry]
```

DescendingIterator in Collections

The **descendingIterator()** is a method in the **Deque interface** (part of the Java Collections Framework). It provides an **iterator** to traverse the elements of a collection in **reverse order** (from the last element to the first).

Where is it Used?

This method is mainly implemented by **Deque** classes like:

- ➔ **ArrayDeque**
- ➔ **LinkedList**

Basic Syntax

```
Java
Iterator<E> descendingIterator();
```

Above code will return an iterator to traverse the collection in reverse order.

Use Case in Real Life:

- ➔ Traversing the history of operations (like a stack of commands).
- ➔ Reversing a queue to process elements from last to first.

Example 1: Using descendingIterator() with ArrayDeque

Real-World Analogy:

Think of a stack of plates. If you want to process plates from the top (last added) to the bottom (first added), descendingIterator() is like taking each plate one by one from the top.

```Java
import java.util.ArrayDeque;
import java.util.Iterator;

public class DescendingIteratorExample {
    public static void main(String[] args) {
        // Creating a deque
        ArrayDeque<String> deque = new ArrayDeque<>();

        // Adding elements
        deque.add("Plate 1");
        deque.add("Plate 2");
        deque.add("Plate 3");

        // Using descendingIterator
        Iterator<String> descIterator = deque.descendingIterator();

        System.out.println("Traversing in reverse order:");
        while (descIterator.hasNext()) {
            System.out.println(descIterator.next());
        }
    }
}
```

Output

```Java
Traversing in reverse order:
Plate 3
Plate 2
Plate 1
```

Example 2: Using descendingIterator() with LinkedList

Real-World Analogy:

Imagine you have a list of tasks completed during the day. If you want to review tasks in reverse order (from the latest to the earliest), descendingIterator() helps.

```java
import java.util.LinkedList;
import java.util.Iterator;

public class DescendingIteratorLinkedList {
    public static void main(String[] args) {
        // Creating a LinkedList
        LinkedList<String> tasks = new LinkedList<>();

        // Adding tasks
        tasks.add("Wake up");
        tasks.add("Brush teeth");
        tasks.add("Have breakfast");

        // Using descendingIterator
        Iterator<String> descIterator = tasks.descendingIterator();

        System.out.println("Tasks in reverse order:");
        while (descIterator.hasNext()) {
            System.out.println(descIterator.next());
        }
    }
}
```

Output:

```java
Tasks in reverse order:
Have breakfast
Brush teeth
Wake up
```

Using For Loop with Index (for Lists): If you need to access elements by index (works only for List implementations like ArrayList and LinkedList), a traditional for loop is suitable. It allows more control when you need to use the index.

Example: For Loop with ArrayList

```Java
import java.util.ArrayList;

public class ForLoopWithIndex {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();
        names.add("melvin");
        names.add("yuvraj");
        names.add("sandeep");

        // Using a regular for loop with index
        for (int i = 0; i < names.size(); i++) {
            System.out.println(names.get(i));
        }
    }
}
```

Output

```Java
melvin
yuvraj
sandeep
```

Fail-Fast vs Fail-Safe in Java Collections: Concurrent Modification or Structural Modification

Java Collections API provides two mechanisms for iterating over data structures: **Fail-Fast** and **Fail-Safe**. Understanding these is important to handle concurrency and modification scenarios effectively.

What is Fail-Fast?

When we try to access the data and at the same time modify the data simultaneously it is known as Concurrent Modification or Structural Modification

- ➔ **Definition**: Fail-Fast iterators immediately throw a ConcurrentModificationException if the structure of the collection is modified while iterating over it.
- ➔ **Mechanism**: The iterator keeps a "modification count" internally, which it checks at each iteration step. If the count changes unexpectedly (due to structural modification), the exception is triggered.

Key Features of Fail-Fast

- ➔ **Applicable Collections**: Most collections in java.util like ArrayList, HashMap, LinkedList, etc., are Fail-Fast.
- ➔ **Trigger Point**: Structural changes such as adding or removing elements (outside the iterator's methods) during iteration.
- ➔ **Real-Time Check**: Fail-Fast iterators actively monitor changes, leading to immediate detection.

Real-World Analogy for Fail-Fast

Imagine a team working on a shared whiteboard. If someone erases or writes something unexpectedly, everyone else stops immediately and raises a flag, saying, *"We can't continue until we resolve this interference!"*

Code Example: Fail-Fast Behavior with for loop

Applicable to collections like ArrayList, LinkedList, etc., but may lead to unpredictable behavior if the structure is modified.

```java
import java.util.ArrayList;

public class ForLoopExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // Using a for loop to iterate
        for (int i = 0; i < list.size(); i++) {
            System.out.println(list.get(i));
              list.add("Date"); // No exception, but can cause
infinite loop or index issues
        }
    }
}
```

Output

```java
Date
Date
Date
Infinite print of Date...
```

Now we can see that the for loop doesn't have Fail-Fast behaviour that is why after each iteration in for loop the value has been added to the list and the list dynamically increases its size so the **i < list.size()** is increasing automatically so the loop is running infinite times

Code Example: Fail-Fast Behavior

```Java
import java.util.ArrayList;
import java.util.Iterator;

public class FailFastExample {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // Using Iterator to traverse the list
        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
            // Modifying the list while iterating
                        list.add("Date");  //  This  triggers
ConcurrentModificationException
        }
    }
}
```

Output

```Java

Apple

Exception in thread "main"
java.util.ConcurrentModificationException
```

Now in above example we can see the addition is not happening and program thrown an `ConcurrentModificationException` exception, so this is the inbuilt Fail-Fast feature of Iterator

Advantages of Fail-Fast

- ➔ **Immediate Feedback**: Alerts developers about unintended modifications, helping to debug faster.
- ➔ **Data Integrity**: Prevents unpredictable outcomes by halting unsafe operations.

Disadvantages of Fail-Fast

- ➔ **Not Thread-Safe**: Even single-threaded modification during iteration causes issues.
- ➔ **No Recovery Mechanism**: Once the exception occurs, iteration stops entirely.

What is Fail-Safe?

Fail-Safe iterators let us safely modify a collection while looping through it without errors. They work on a **copy** of the collection instead of the original one. We'll find this in special collections like CopyOnWriteArrayList or ConcurrentHashMap which are present in `java.util.concurrent` package. However, changes made during the loop won't show up right away.

- ➔ **Definition**: Fail-Safe iterators do not throw exceptions when the collection is modified during iteration.
- ➔ **Mechanism**: They work on a copy of the collection (often called a "snapshot"), so structural changes do not affect the ongoing iteration.

Key Features of Fail-Safe

- ➔ **Applicable Collections**: Collections in the java.util.concurrent package, like ConcurrentHashMap, CopyOnWriteArrayList.
- ➔ **No Exception**: Modifications during iteration are safely handled without throwing errors.
- ➔ **Snapshot Behavior**: Iteration reflects the state of the collection at the time the iterator was created.

Real-World Analogy for Fail-Safe

Imagine a chef writing down a recipe from a shared chalkboard. Even if someone else erases or rewrites the chalkboard, the chef continues to work with the recipe they copied earlier.

Code Example: Fail-Safe Behavior

```Java
import java.util.concurrent.CopyOnWriteArrayList;

public class FailSafeExample {
    public static void main(String[] args) {
        CopyOnWriteArrayList<String> list = new
CopyOnWriteArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // Using Iterator to traverse the list
        for (String fruit : list) {
            System.out.println(fruit);
            // Modifying the list while iterating
            list.add("Date");
        }

        System.out.println("Final List: " + list); // No
exception, all elements included
    }
}
```

Output

```Java
Apple
Banana
Cherry
```

```
Final List: [Apple, Banana, Cherry, Date]
```

In this above example we can see that either no infinite loop or no `ConcurrentModificationException as been thrown, but the program executed normally`

Advantages of Fail-Safe

- ➔ **Thread-Safe**: Suitable for concurrent programming.
- ➔ **Continues Execution**: Allows iteration to complete without disruptions.

Disadvantages of Fail-Safe

- ➔ **Increased Memory Use**: Maintaining a snapshot can be resource-intensive.
- ➔ **Stale Data**: Iteration reflects the state at the time of creation, so new changes are not visible.

Comparison Table: Fail-Fast vs. Fail-Safe

Aspect	Fail-Fast	Fail-Safe
Collection Type	ArrayList, HashMap, etc.	ConcurrentHashMap, CopyOnWriteArrayList
Mechanism	Monitors changes using a modification count	Iterates over a copy (snapshot) of the collection

Exception	ConcurrentModificationException	No exceptions
Thread-Safe	No	Yes
Performance	Faster, no extra memory required	Slower, requires additional memory

When to Use?

1. **Fail-Fast**: For single-threaded applications or scenarios where structural modifications during iteration are unlikely.
2. **Fail-Safe**: For multi-threaded environments where concurrent modifications are expected.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan